NoC Router Verification Notes

--- 1/7/2026 ---

DUT (Design Under Test)

Το DUT που πρόκειται να επαληθευτεί είναι ένας δρομολογητής (Router), σκοπός του οποίου είναι η δρομολόγηση πακέτων μεταξύ κόμβων σε ένα δίκτυο NoC (Network on Chip).

- Το δίκτυο έχει τοπολογία πλέγματος με παραμετροποιημένο μέγεθος διαστάσεων (π.χ. 2x2 με 4 κόμβους, 4x4 με 16 κόμβους, κλπ).
- Ο Router έχει 5 θύρες εισόδου-εξόδου, Βόρεια, Νότια, Ανατολική, Δυτική και Τοπική (ή North, South, East, West και Local).
- Η βασική μονάδα πληροφορίας που αποστέλλονται και λαμβάνονται στο δίκτυο είναι το flit. Το μεγεθός του είναι παραμετροποιημένο.
- Τα πακέτα που δρομολογούνται στο δίκτυο αποτελούνται από ένα έως τέσσερα flits.
- Η ροή δεδομένων ελέγχεται μέσω Εικονικών Καναλιών (Virtual Channels).
- Χρησιμοποιείται πολιτική credit-based backpressure mechanism ώστε κάθε Router που πρόκειται να στείλει δεδομένα να γνωρίζει αν υπάρχει διαθέσιμος χώρος στον Router λήψης.
- Χρησιμοποιούνται δύο τύποι Virtual Channel για την αποφυγή των Deadlocks. Αυτά είναι τα Adaptive VCs που εφαρμόζουν ελάχιστη προσαρμοστική δρομολόγηση, και Escape VCs που εφαρμόζουν XY Routing.
- Εφαρμόζεται συντηρητική επαναδέσμευση των Virtual Channels, δηλαδή κάθε VC μπορεί κάθε στιγμή να είναι δεσμευμένο από το πολύ ένα πακέτο.
- Η μεταφορά των πακέτων γίνεται σε 4 στάδια λειτουργίας:
    - RC: Υπολογισμός των υποψήφιων θυρών εξόδου.
    - VA: Ανάθεση ενός downstream VC για το πακέτο.
    - SA: Επιλογή του flit το οποίο οποίο θα διασχίσει τον μεταγωγέα.
    - ST: Η φυσική διέλευση των flits από τον μεταγωγέα.

Sequence Item

- Η βασική μονάδα πληροφορίας sequence item (ή transaction), θα είναι το flit.
- Για μέγεθος flit, FLIT_WIDTH = 64 bits, η δομή του είναι η εξής:

Θέση Bits	Μέγεθος	Περιεχόμενο	Περιγραφή
[63:62]	2 bits	flit_label	Το είδος του flit (HEAD, BODY, TAIL, HEADTAIL)
[61]	1 bit	vc_id	Το είδος του εικονικού καναλιού που θα σταλεί (Escape VC ή Adaptive VC)
[60:57] (Περίπτωση Head Flit)	4 bits	x_dest , y_dest	Συντεταγμένες προορισμού
[56:0] (Περίπτωση Head Flit)	57 bits	head_pl	Ωφέλιμα δεδομένα
[60:0] (Περίπτωση Body flit)	61 bits	bt_pl	Ωφέλιμα δεδομένα

* Τα σήματα *is_valid, is_allocatable* είναι σήματα ελέγχου, κι έτσι δεν θα αποτελούν μέρος του sequence item.

Interface

- Οι θύρες του Router αποτελούνται από 10 σετ διεπαφών, ένα ζευγάρι upstream και downstream για κάθε κατεύθυνση (Βόρεια, Νότια, Ανατολική, Δυτική και Τοπική).
- Οι upstream θύρες είναι αυτές από τις οποίες πρόκειται να στείλει δεδομένα ο Router ενώ οι downstream αυτές από τις οποίες θα δεχτεί δεδομένα.
- Στις upstream τα σήματα εξόδου είναι το *data* που είναι τύπου flit (όπως περιγράφηκε παραπάνω), και το σήμα *is_valid* ενός bit που υποδεικνύει πως τα δεδομένα προς αποστολή είναι έγκυρα. Τα σήματα εισόδου είναι το *credits* μεγέθους 2 bits που υποδεικνύει πόσος χώρος υπάρχει διαθέσιμος στον VC λήψης, και το σήμα *is_allocatable* που δείχνει στον Router ποιο VC είναι διαθέσιμο για παραλαβή νέου πακέτου.
- Στις downstream τα σήματα εισόδου-εξόδου είναι ακριβώς αντίστροφα από τις upstream.

Αρχιτεκτονική των Agents

Κάθε μία από τις 5 κατευθύνσεις του Router χρησιμοποιείται για να στείλει και για να λάβει δεδομένα, μέσω των upstream και downstream interface. Για αυτό θα χρησιμοποιηθούν 10 Agents, 5 Master και 5 Slave, οι οποίοι θα αλληλεπιδρούν με τα interfaces.

- Οι Master Agents θα στέλνουν δεδομένα στα downstream interfaces του DUT. Ο Sequencer θα στέλνει ακολουθίες από flits στον Driver. Ο Driver ανάλογα με τη διαθεσιμότητα του Router (κοιτώντας τα σήματα *credits, is_allocatable*), θα στέλνει flits σε αυτόν οδηγώντας ανάλογα τα σήματα *data* και *is_valid*. Ο monitor παρακολουθεί αυτά τα σήματα και τα στέλνει στο scoreboard.
- Οι Slave Agents θα λαμβάνουν τα δεδομένα από τα upstream interfaces του DUT. Ο Driver είναι υπεύθυνος για να λαμβάνει τα flits από το DUT και με βάση αυτά να ανανεώνει τα σήματα *credits* και *is_allocatable.* Ο Monitor στέλνει όλα τα σήματα στο scoreboard.

Σκοπός των Agents είναι να προσομοιώνουν τη συμπεριφορά των γειτονικών Routers του DUT, κι έτσι να προσομοιωθεί η συμπεριφορά του δικτύου γύρω από αυτό.

Test Plan

Για την σωστή επαλήθευση της λειτουργίας του DUT πρέπει να δοκιμαστούν πολλά διαφορετικά και στοχευμένα σενάρια. Παρακάτω παρουσιάζονται κάποια βασικά σενάρια:

- Test 1: Αποστολή ενός πακέτου ενός flit (τύπου HEADTAIL) με τυχαίο payload και τυχαίο κόμβο προορισμού. Σκοπός είναι να επιβεβαιωθεί αν ο Router υπολογίζει σωστά την θύρα εξόδου, και αν μεταφέρει ακέραια την πληροφορία στην έξοδο.
- Test 2: Αποστολή ενός πακέτου πολλαπλών flits (με HEAD, BODY TAIL), με τυχαίο payload και κόμβο προορισμού. Ο σκοπός είναι ίδιος με του Test 1.
- Test 3: Ταυτόχρονη αποστολή πακέτων από διαφορετικούς Master Agents με κόμβους προορισμούς τέτοιους ώστε να καταλήγουν στην ίδια θύρα εξόδου. Εδώ σκοπός είναι να δρομολογηθούν όλα τα πακέτα με την κατάλληλη προτεραιότητα και να μην χαθούν δεδομένα.
- Test 4: Συνεχόμενη αποστολή πακέτων προς μία θύρα εξόδου. Σε αυτό το σενάριο θα μειώνονται και κάποια στιγμή θα μηδενιστούν τα *credits* στην είσοδο του upstream VC. Τότε το DUT θα πρέπει να σταματήσει να στέλνει δεδομένα, δηλαδή θα πρέπει το σήμα *is_valid* να γίνει 0.
- Test 5: Αποστολή flits με λανθασμένη σειρά, για παράδειγμα αποστολή TAIL flit χωρίς να προηγηθεί HEAD ή αποστολή δύο HEAD flits στη σειρά. Σε αυτήν την περίπτωση θα πρέπει το σήμα *error_o* του DUT να πάρει την τιμή 1.

--- 8/7/2026 ---

Λειτουργία των σημάτων ελέγχου

- Η διεπαφή για κάθε μία από τις 5 κατευθύνσεις του router χωρίζεται σε upstream και downstream.
- Το upstream σετ θυρών χρησιμοποιείται για να στέλνει ο router πακέτα σε γειτονικούς routers, ενώ το downstream σετ θυρών είναι για να δέχεται πακέτα από γειτονικούς routers.
- Στην περίπτωση του downstream τα σήματα εισόδου είναι τα *data*,τύπου *flit_t* και *is_valid,* σήμα ελέγχου ενός bit, ενώ στην έξοδο είναι τα σήματα ελέγχου *credits* τύπου credits_t, και is_allocatable μεγέθους όσο είναι ο αριθμός των καναλιών, δηλαδή 2.
- Στα upstream τα σήματα είναι τα ίδια αλλά με αντίθετες κατευθύνσεις.
- Το σήμα *data* μεταφέρει την πληροφορία, τα flits που δρομολογούνται μεταξύ των routers.
- Το σήμα *is_valid* είναι το σήμα ελέγχου που όταν είναι 1 υποδεικνύει πως ένα flit είναι έτοιμο να σταλεί / δεχθεί και πρέπει να καταχωρηθεί από τον δέκτη, ενώ αν είναι 0, ο δέκτης δεν πρέπει να διαβάσει τα δεδομένα *data*.
- Το σήμα *credits* είναι το σήμα ελέγχου που δηλώνει πως ελευθερώθηκε θέση σε κάποιο από τα κανάλια της θύρας. Έχει μέγεθος 2 bits, ένα για κάθε κανάλι. Όταν τα bits παίρνουν την τιμή 1, δηλώνουν ότι ελευθερώθηκε μία θέση στον buffer σε αυτόν τον κύκλο ρολογιού. Αυτό χρειάζεται διότι οι routers έχουν εσωτερικούς μετρητές για να γνωρίζουν πόσες ελεύθερες θέσεις έχουν στους buffers τους οι γειτονικοί routers. Διαβάζοντας το σήμα *credits* οι μετρητές αυτοί αυξάνονται κατά 1, ενώ μειώνονται κατά 1 κάθε φορά που αποστέλλεται flit από αυτούς. Πχ στην upstream βόρεια θύρα ενός router, λαμβάνεται το σήμα *credits = 2'b01.* Έτσι ο router ενημερώνεται πως το escape κανάλι της νότιας θύρας του γειτονικού του router, ελευθέρωσε μια θέση στον buffer του, επομένως θα ενημερώσει τον εσωτερικό μετρητή του ότι οι κενές θέσεις σε αυτό το κανάλι αυξήθηκαν κατά 1 σε αυτόν τον κύκλο ρολογιού.
- Το σήμα ελέγχου *is_allocatable* δείχνει στον πομπό την διαθεσιμότητα του δέκτη. Αποτελείται από 2 bits, ένα για κάθε τύπου κανάλι, οπότε αν το αντίστοιχο bit κάθε καναλιού είναι 1, δηλώνει πως το κανάλι είναι διαθέσιμο για να δεχθεί νέο πακέτο. Πχ σε έναν router στην ανατολική μεριά στην downstream διεπαφή, το σήμα εξόδου *is_allocatable* είναι ίσο με 2'b10. Αυτό σημαίνει πως μόνο το adaptive ανατολικό κανάλι του router μπορεί να δεχτεί νέο πακέτο. Αυτό το σήμα λαμβάνεται από τον γειτονικό router στην δυτική του upstream διεπαφή.
- Το σήμα εξόδου *error_o* του router, που δεν ανήκει σε κάποιο interface, ενημερώνει όταν εντοπίζονται παράνομες συμπεριφορές. Πιο συγκεκριμένα, τα σενάρια όπου το *error_o* γίνεται ίσο με 1 είναι τα εξής:
  - Όταν ξεκινάει να στέλνεται νέο πακέτο με το πρώτο flit να είναι τύπου BODY ή TAIL, αντί για HEAD ή HEADTAIL.

- Όταν επιχειρείται να εισαχθεί flit τύπου HEAD σε ένα ήδη δεσμευμένο κανάλι που δεν έχει αποδεσμευτεί ακόμα.
- Όταν το κανάλι βρίσκεται σε κατάσταση IDLE ή RC, και στέλνεται σήμα vc_valid_i που δηλώνει ανάθεση downstream καναλιού.
- Όταν επιχειρείται να διαβαστεί (read_cmd = 1) ο buffer όταν είναι κενός, δηλαδή βρίσκεται στο στάδιο IDLE ή RC.

--- 15/7/2026 ---

Σήμα error_o: Συμπεριφορά και διευκρινίσεις

Το σήμα εξόδου *error_o* του router, που δεν ανήκει σε κάποιο interface, ενημερώνει όταν εντοπίζονται παράνομες συμπεριφορές. Το module στο οποίο υπολογίζεται η τιμή του είναι το vc_buffer. Το vc_buffer περιέχει ένα FSM τεσσάρων καταστάσεων (IDLE, RC, VA, SA), όπου ανάλογα με την κατάσταση που βρίσκεται και ποιες τιμές έχουν κάποια εσωτερικά σήματα τίθεται η τιμή του 0 ή 1. Παρακάτω εξηγούνται η περιπτώσεις όπου το σήμα σφάλματος θα τεθεί στο 1:

- Στην κατάσταση IDLE, όταν το vc_valid_i ή το read_cmd είναι 1. Δηλαδή όταν το κανάλι είναι ανενεργό και άδειο, και ταυτόχρονα και είτε ζητείται να διαβαστούν δεδομένα από αυτό είτε του ανατίθεται downstream κανάλι (vc_valid_i = 1), τότε το error γίνεται 1.
- Στην κατάσταση RC, όταν ο τύπος του flit λήψης είναι BODY ή TAIL ή κάποιο από τα σήματα vc_valid_i και read_cmd είναι 1, σηκώνεται το σήμα error. Στην κατάσταση RC το κανάλι έχει μόλις δεχθεί το πρώτο flit του πακέτου και υπολογίζονται οι πιθανές δρομολογήσεις του. Επομένως δεν πρέπει να είναι BODY ή TAIL. Επίσης δεν πρέπει να διαβαστούν flits από αυτό αφού δεν έχει ολοκληρωθεί η δρομολόγηση, και ούτε πρέπει να του ανατεθεί downstream κανάλι στο στάδιο RC.
- Στην κατάσταση VA, αν το read_cmd είναι 1 ή αν το write_cmd είναι 1 και ταυτόχρονα είναι ο τύπος HEAD ή HEADTAIL ή το end_packet είναι 1, τότε σηκώνεται το error. Σε αυτό το στάδιο υπολογίζεται ποιο θα είναι το downstream κανάλι που θα δεχθεί το πακέτο. Επομένως δεν πρέπει να διαβαστούν ακόμα τα flits από το κανάλι, όπως επίσης δεν γίνεται να γραφούν στο κανάλι flits τύπου HEAD ή HEADTAIL, ή να γραφούν δεδομένα ενώ τελευταίο flit που εισήχθη ήταν το τελευταίο του πακέτου.
- Στην κατάσταση SA, όταν το vc_valid_i είναι 1, ή όπως προηγουμένως αν το write_cmd είναι 1 και ταυτόχρονα ο τύπος είναι είτε HEAD είτε HEADTAIL, ή το end_packet είναι 1, σηκώνεται το error. Στην κατάσταση SA επιλέγεται το ποιο flit θα διασχίσει το μεταγωγέα. Επομένως δεν πρέπει να ξαναγίνει ανάθεση downstream. Επίσης για εγγραφή νέων flits ισχύει ότι και στο VA.
- Όταν το FSM βρίσκεται σε κάποια μη γνωστή κατάσταση (δηλαδή καμία από τις παραπάνω) υψώνεται και πάλι το σήμα σφάλματος.

* Επεξήγηση λειτουργίας εσωτερικών σημάτων του vc_buffer

- vc_valid_i: Υποδηλώνει επιτυχή ανάθεση downstream καναλιού
- read_cmd: Εντολή επιτυχούς ανάγνωσης και εξαγωγής flit
- write_cmd: Εντολή επιτυχούς εγγραφής flit
- end_packet: Υποδηλώνει πως εισήχθη το τελευταίο flit ενός πακέτου

--- 17/7/2026 ---

Αναλυτική αρχιτεκτονική των Agents

Κάθε router έχει 5 αμφίδρομες θύρες (North, South, East, West, Local) μέσω των οποίων επικοινωνεί και στέλνει δεδομένα στους γειτονικούς routers. Οι διεπαφές της κάθε θύρας χωρίζονται σε upstream και downstream. Οι upstream διεπαφές χρησιμοποιούνται όταν ο router θέλει να στείλει δεδομένα, ενώ οι downstream όταν θέλει να λάβει.

Στο περιβάλλον επαλήθευσης UVM οι Agents είναι οντότητες εξωτερικές του DUT, που στέλνουν και λαμβάνουν σήματα από αυτό μέσω των διεπαφών του. Η παρουσία των Agents προσομοιώνει την ύπαρξη των γειτονικών router σε ένα NoC. Έτσι, θα υπάρχουν δύο τύποι Agents, οι Master που θα στέλνουν πακέτα στο DUT μέσω των downstream θυρών, και οι Slave που θα λαμβάνουν σήματα από το DUT μέσω των upstream θυρών. Επομένως θα υπάρχουν συνολικά 10 Agents, 5 Master και 5 Slave.

Master Agent

Ο ρόλος αυτού του Agent είναι να στέλνει πακέτα προς το DUT, προσομοιώνοντας τον router εκπομπής. Τα σήματα που δίνει στην έξοδό του είναι τα *data* και *is_valid,* που είναι το flit προς αποστολή και το σήμα επιβεβαίωσης. Στις εισόδους του δέχεται τα σήματα *credits* και *is_allocatable*, δηλαδή το σήμα που ενημερώνει τον εσωτερικό μετρητή του Agent για κενές θέσεις στο DUT, και το σήμα διαθεσιμότητας δέσμευσης των καναλιών του DUT. Το sequence item που επιλέχθηκε έχει την μορφή του flit, επομένως τα υπόλοιπα σήματα (ελέγχου), πρέπει να χειρίζονται με λογική υλοποιημένη στον Agent. Ο Master Driver θα πρέπει να ελέγχει αν υπάρχουν διαθέσιμες θέσεις, παρακολουθώντας τα credits, και αν τα κανάλια του DUT μπορούν να δεσμευτούν (*is_allocatable*), για να σηκώσει το σήμα *is_valid* και να στείλει flits. Ο Driver δεν ασχολείται με την εγκυρότητα των flits, υποθέτει πως ο sequencer τα στέλνει σωστά.

Slave Agent

Ο ρόλος του Slave είναι να λαμβάνει πακέτα από το DUT, προσομοιώνοντας τον router λήψης. Στις εισόδους του δέχεται τα σήματα *data* και *is_valid,* και στις εξόδους του έχει τα *credits* και *is_allocatable.* Ο Driver οδηγεί τα σήματα που δείχνουν στο DUT τη διαθεσιμότητα του Agent. Για να μην δημιουργήσει συμφόρηση, πρέπει ανά κάποιο χρονικό διάστημα να “καταναλώνει” flits στέλνοντας σήμα *credits* στην έξοδο. Επίσης πρέπει να χειρίζεται το *is_allocatable,* οδηγώντας το στο 0 όταν δέχεται ένα νέο πακέτο, και στο 1 όταν ολοκληρώνεται η αποστολή του πακέτου.

--- 27/7/2026 ---

Δομή κώδικα Master Driver

Ο Master Driver είναι το component του Agent που επικοινωνεί με το DUT (downstream interface) διαβάζοντας τις εξόδους του και ανάλογα οδηγεί τις δικές του εξόδους. Στην παρούσα υλοποίηση δημιουργήθηκε η κλάση *router_master_driver* οι οποία κάνει extend την *uvm_driver* με παράμετρο το *router_seq_item*.

Τα σήματα που δέχεται από το DUT (is_allocatable, credits) είναι παλμοί διάρκειας ενός κύκλου. Δηλαδή το DUT στέλνει ένα σήμα credits κάθε φορά που στέλνει ένα flit, δηλαδή ελευθερώνεται μία θέση στους buffer του. Το σήμα *is_allocatable* από το DUT στέλνεται όταν φεύγει από τους buffers του το τελευταίο flit ενός πακέτου, ενημερώνοντας έτσι ότι είναι ελεύθερο για δέσμευση από άλλο πακέτο. Εφόσον αυτά τα σήματα έχουν διάρκεια ενός κύκλου, πρέπει ο driver να έχει εσωτερικές μεταβλητές που θα κρατάνε την κατάσταση του DUT ώστε να γνωρίζει πότε θα υπάρχει διαθεσιμότητα για αποστολή flits και πακέτων. Αυτόν τον σκοπό εξυπηρετούν τα εσωτερικά σήματα *vc_credits* και *is_vc_allocatable.* Το πρώτο κρατάει τον αριθμό των ελεύθερων θέσεων στο κάθε κανάλι του DUT, και το δεύτερο δείχνει (όταν είναι 1) το αν το κάθε κανάλι είναι διαθέσιμο να δεσμευτεί με νέο πακέτο. Και τα δύο ενημερώνονται σε κάθε χτύπο του ρολογιού σύμφωνα με τα σήματα εισόδου. Επίσης η κλάση περιέχει ένα βασικό co nstructor και την συνάρτηση build_phase.

Στο task run_phase γίνονται όλοι οι υπολογισμοί ώστε να διαβαστούν οι είσοδοι και να οδηγηθούν σωστά οι έξοδοι. Το μπλοκ είναι τυλιγμένο με μια forever loop, καθώς η λειτουργία του θα επαναλαμβάνεται συνεχώς για όλη τη διάρκεια του simulation time. Στην αρχή περιμένει μέχρι να σταματήσει το reset, και αφού γίνει αυτό, μέσω του *reset_driver_state()* task οδηγεί τα σήματα εξόδου στο μηδέν, και αρχικοποιεί τα εσωτερικά σήματα. Στη συνέχεια χρησιμοποιείται μια fork...join_any χάρη στην οποία τρέχουν παράλληλα δύο threads. Το πρώτο τρέχει το *drive_loop()* task, και το δεύτερο περιμένει πότε το *rst_n* θα πάει στο μηδέν. Αυτή η τεχνική χρησιμοποιείται για να μπορεί το ασύγχρονο reset να μπορεί διακόψει τη λειτουργία της οδήγησης των σημάτων οποιαδήποτε στιγμή τεθεί στο 0.

Στο *drive_loop()* γίνεται όλη η διαδικασία της ανάγνωσης σημάτων, ενημέρωση μεταβλητών, και απόφαση αποστολής ή μη των flits οδηγώντας ανάλογα τις εξόδους. Το task είναι τυλιγμένο με μια forever loop και όλες οι λειτουργίες μέσα της γίνονται πάνω στον χτύπο του ρολογιού. Αρχικά, με τη χρήση της συνάρτησης update_flow_control() διαβάζονται τα σήματα εισόδου που αφορούν τις ελεύθερες θέσεις και την διαθεσιμότητα δέσμευσης των καναλιών του DUT και ενημερώνονται ανάλογα οι εσωτερικές μεταβλητές. Στη συνέχεια αν δεν υπάρχει transaction σε εξέλιξη ζητά από τον sequencer ένα νέο item. Εάν μετά δεν υπάρχει νέο item προς αποστολή το *is_valid* οδηγείται στο 0, αφήνοντας έτσι το DUT να γνωρίζει πως δεν του στέλνεται νέο flit και επαναλαμβάνεται η ίδια διαδικασία από την αρχή στον επόμενο χτύπο του ρολογιού. Αν όμως ο sequencer δίνει νέο item στον driver, η ροή πάει ως εξής. Ελέγχεται με την συνάρτηση check_and_assign_vc() αν το DUT μπορεί να δεχτεί το παρών flit. Αν το παρόν flit είναι το πρώτο ενός πακέτου (HEAD ή HEADTAIL) κοιτάει τον τύπο καναλιού του flit (ADAPTIVE ή ESCAPE) και αν το αντίστοιχο κανάλι είναι allocatable και έχει χωρητικότητα θεωρείται εφικτή η αποστολή του

flit. Επίσης χειρίζεται και η περίπτωση που το ADAPTIVE κανάλι δεν είναι διαθέσιμο αλλά είναι το ESCAPE, επομένως ο τύπος καναλιού του flit, σύμφωνα με το πρωτόκολλο. Στην περίπτωση που το flit που ελέγχεται δεν είναι το πρώτο του πακέτου τότε αρχικά πρέπει να ενημερωθεί ο τύπος καναλιού του, για την περίπτωση που το HEAD άλλαξε τύπο όπως αναφέρθηκε προηγουμένως. Στη συνέχεια το μόνο που χρειάζεται να ελεγχθεί είναι ο διαθέσιμος χώρος στο κανάλι προορισμού, κι εφόσον υπάρχει η αποστολή του flit δηλώνεται εφικτή. Εφόσον γίνει αυτή η διαδικασία, πίσω στο *drive_loop* εφόσον αποφασιστεί η αποστολή flit, το *is_valid* οδηγείται στο 1, και το *data* παίρνει τα δεδομένα του transaction, κι ενημερώνονται τα εσωτερικά σήματα του driver. Στην περίπτωση που το check_and_assign_vc() επιστρέψει 0, δηλαδή δεν είναι εφικτή η αποστολή flit, δεν γίνονται τα παραπάνω και το is_valid οδηγείται στο μηδέν.

Όλη αυτή η διαδικασία επαναλαμβάνεται σε κάθε χτύπο του ρολογιού είτε μέχρι το τέλος του simulation time, είτε μέχρι να διακοπεί από reset, όπου και θα ξεκινήσει ξανά από την αρχή.

Αυτή είναι η τυπική και αναμενόμενη συμπεριφορά του driver, ώστε να επικοινωνεί με το DUT σύμφωνα με το πρωτόκολλο. Όμως θα χρειαστεί να δοκιμαστεί η λειτουργία του DUT και όταν το εξωτερικό του περιβάλλον έχει μη αναμενόμενη συμπεριφορά. Για αυτόν τον λόγο, σε μελλοντικά βήματα θα τροποποιηθεί με τρόπο ώστε όταν ένα test το επιβάλλει, να λειτουργεί με “ανορθόδοξο” τρόπο και “παράνομη” συμπεριφορά που δεν συμμορφώνεται με το πρωτόκολλο. Για παράδειγμα να στέλνει flits ενώ γνωρίζει πως οι buffers του DUT είναι πλήρης, ή να ξεκινά η αποστολή νέου πακέτου όταν το κανάλι προορισμού δεν είναι διαθέσιμο για δέσμευση.

--- 12/8/2026 ---

Δομή κώδικα Slave Driver

Ο Slave Driver επικοινωνεί με το upstream interface του DUT. Ο σκοπός του είναι να δέχεται τα πακέτα μέσω των σημάτων *data* και *is_valid,* και να τα "καταναλώνει" στέλνοντας πίσω στο DUT τα σήματα *credits* και *is_allocatable,* που ενημερώνουν το DUT για το αν ελευθερώθηκαν θέσεις στα "κανάλια" του Agent, και αν είναι διαθέσιμα για να δεχθούν νέο πακέτο. Ο Driver αυτός υλοποιείται στην κλάση *router_slave_driver.*

Η κλάση περιέχει τα εσωτερικά σήματα *pending_credits* και *pending_allocatable.* Το *pending_credits* μετράει πόσα flits έχει δεχθεί και δεν έχουν "καταναλωθεί", το οποίο είναι ο αριθμός των credits που οφείλει να στείλει στο DUT για να ενημερώσει τους μετρητές για τις διαθέσιμες θέσεις στα "κανάλια" του Agent. Το pending_allocatable υποδεικνύει αν ο Agent δέχθηκε TAIL ή HEADTAIL flit, ώστε να γνωρίζει να στείλει το σήμα is_allocatable στο DUT όταν θα καταναλωθεί το συγκεκριμένο flit.

Στο task run_phase υλοποιείται η λογική της συμπεριφοράς του driver. Το εσωτερικό του task είναι τυλιγμένο σε forever block, εφόσον ο driver δεν θα σταματήσει ποτέ να λειτουργεί κατά τη διάρκεια του simulation time. Στην αρχή γίνεται η αρχικοποίηση των εσωτερικών σημάτων και των σημάτων εξόδου με τη χρήση του task *reset_driver_state()*. Στην συνέχεια αφού το reset ανέβει στο 1, ξεκινάει η παράλληλη εκτέλεση των tasks *monitor_incoming_flits(), drive_vc_flow_control(ESCAPE), drive_vc_flow_control(ADAPTIVE), και wait(vif.rst_n === 1'b0) ,* με τη χρήση της fork...join_any. Η wait χρησιμοποιείται σε συνδυασμό με την εντολή disable fork για τον χειρισμό του on the fly reset.

Το *monitor_incoming_flits()* είναι υπεύθυνο για την ανάγνωση των εισόδων *is_valid* και *data,* και με βάση αυτές ενημερώνει τα εσωτερικά σήματα *pending_credits* και pending_allocatable. Η ανάγνωση και ενημέρωση γίνεται ακριβώς πάνω στον χτύπο του ρολογιού, χωρίς την χρήση του clocking block για την αποφυγή race conditions, όπως θα εξηγηθεί σε επόμενη παράγραφο.

Το *drive_vc_flow_control(vc_class_t vc_id)* αναλαμβάνει να οδηγήσει τα σήματα εξόδου *credits* και *is_allocatable.* Σύμφωνα με το αν υπάρχουν flits προς κατανάλωση (*pending_credits*) σηκώνει το σήμα *credits* για έναν κύκλο ρολογιού, και επίσης σηκώνει το *is_allocatable* αν με την αποστολή του credit ολοκληρώνεται η κατανάλωση ενός ολόκληρου πακέτου, και ανάλογα με αυτές τις δράσεις ενημερώνει και τους αντίστοιχους εσωτερικούς μετρητές. Μια ιδιαιτερότητα αυτού του task είναι η τυχαία καθυστέρηση της αποστολής των credits. Με σκοπό την προσομοίωση διαφορετικών βαθμών "κίνησης" του δικτύου, η αποστολή του flit γίνεται με μια τυχαία καθυστέρηση της οποίας το εύρος καθορίζεται από το Agent Configuration Object. Το task τρέχει ξεχωριστά και παράλληλα για κάθε ένα από τα Virtual Channels. Αυτές η ενέργειες γίνεται σε χτύπο ρολογιού με την χρήση του clocking block.

Η επιλογή χρήσης και μη χρήσης του clocking block σε κάθε task γίνεται για την αποφυγή των race conditions σε περίπτωση ταυτόχρονης πρόσβασης στους "πόρους" *pending_credits* και *pending_allocatable.* Έτσι οι ενέργειες του *monitor_incoming_flits(),* γίνονται πριν τα *drive_vc_flow_control(…).*

Τα tasks τρέχουνε παράλληλα σε διαφορετικά threads μέχρι να διακοπεί η λειτουργία τους από το reset. Τότε αρχικοποιούνται όλες οι τιμές και μπλοκάρεται η εκτέλεση, μέχρι να κατέλθει το reset και να ξαναρχίσουν τα threads.

Monitor

Ο monitor εκτελεί μια πολύ απλή και βασική λειτουργία, να καταγράφει τις εισόδους και εξόδου του DUT. Σε κάθε χτύπο του ρολογιού, αν το δεν βρίσκεται σε κατάσταση reset και αν συμβαίνει κάποια αποστολή flit (*is_valid == 1*), τότε καταγράφει το σήμα data του DUT, το μετατρέπει σε transaction, και το στέλνει σε ένα analysis port.

Agent Configuration Object

Αυτή η κλάση χρειάζεται για την ρύθμιση των παραμέτρων του κάθε Agent. Περιέχει μεταβλητές που υποδεικνύουν τον τύπο του Agent (Master/Slave, Active/Passive), όπως επίσης και το εύρος της καθυστέρησης κατανάλωσης flits για τον slave driver. Επίσης μέσω αυτού γίνεται και η δημιουργία των virtual interfaces στα components των Agents.

Agent

Στην κλάση αυτή δημιουργούνται όλα τα προηγούμενα στοιχεία. Αν ο τύπος είναι Master, δημιουργείται Master Driver και Sequencer και συνδέονται μεταξύ τους. Αν είναι Slave, δημιουργείται Slave Driver. Σε κάθε περίπτωση δημιουργείται ένας Monitor και το analysis port του συνδέεται με το analysis port του Agent.

--- 8/9/2026 ---

Η πρώτη ολοκληρωμένη μορφή του Testbench

Για να διαπιστώσουμε αν τα components που έχουμε αναπτύξει μέχρι στιγμής είναι σωστά και δουλεύουν, δημιουργείται ένα πρώτης και απλής μορφής testbench και test ώστε να γίνει compile του κώδικα και να τρέξει το πρώτο simulation. Για τον σκοπό αυτό αναπτύσσονται τα environment, Sequence, Test, και Top Level Module.

Environment

Εδώ δημιουργούνται οι 10 Agents, 5 Master, 5 Slave, ένα ζευγάρι για κάθε κατεύθυνση του DUT.

Sanity Sequnece

Αυτή το Sequence δημιουργήθηκε για το πιο απλό και αρχικό test. Δημιουργεί μόνο ένα transaction, ένα flit με label HEADTAIL, vc_id = 0 (ESCAPE VC), και τα υπόλοιπα πεδία, τυχαιοποιημένα.

Sanity Test

Το test αυτό δημιουργεί τα κατάλληλα configuration objects για τους Agents, και χρησιμοποιεί το Sanity Sequence.

Top Level

Το υψηλότερο module στην ιεραρχία του UVM Testbench. Εδώ γίνεται η δημιουργία του ρολογιού, του reset, του DUT (router), των interfaces, και η σύνδεσή τους με το UVM Config DB ώστε να επικοινωνεί με το test.

--- 18/9/2026 ---

Πρώτη επιτυχής εκτέλεση του Sanity Test

Η επιτυχία και η μελέτη των αποτελεσμάτων αυτού του test είναι κρίσιμη για να διαπιστωθεί η σωστή λειτουργία του μέχρι στιγμής αναπτυγμένου testbench, και να εντοπισθούν τυχόν λάθη σε αυτό.

Αρχικά, η θέση του DUT router στο πλέγμα καθορίζεται από τις παραμέτρους *TB_X_CURRENT, TB_Y_CURRENT,* οι οποίες καθορίζονται στο Makefile, και στην συγκεκριμένη περίπτωση παίρνουν τις τιμές (1,1) . Το Test δημιουργεί τους 10 Agents, Master και Slave. Επίσης θέτει την καθυστέρηση “κατανάλωσης” flit των Slave Agents στον 1 κύκλο ρολογιού.

Στέλνει ένα flit από τον LOCAL Master Agent. Το flit αυτό δημιουργείται στο Sanity Sequence με τις παρακάτω τιμές:

- flit_label = HEADTAIL;
- vc_id = 0;
- x_dest = 0;
- y_dest = 1;
- payload = random

Επομένως σύμφωνα με την τοπολογία του πλέγματος του NoC, αφού η αφετηρία του πακέτου είναι η θέση (1,1) και ο προορισμός το (0,1), το DUT πρέπει να στείλει αυτό το flit στην WEST θύρα του. Στην επόμενη σελίδα φαίνονται οι κυματομορφές από την εκτέλεση αυτού του test στο Xcelium.

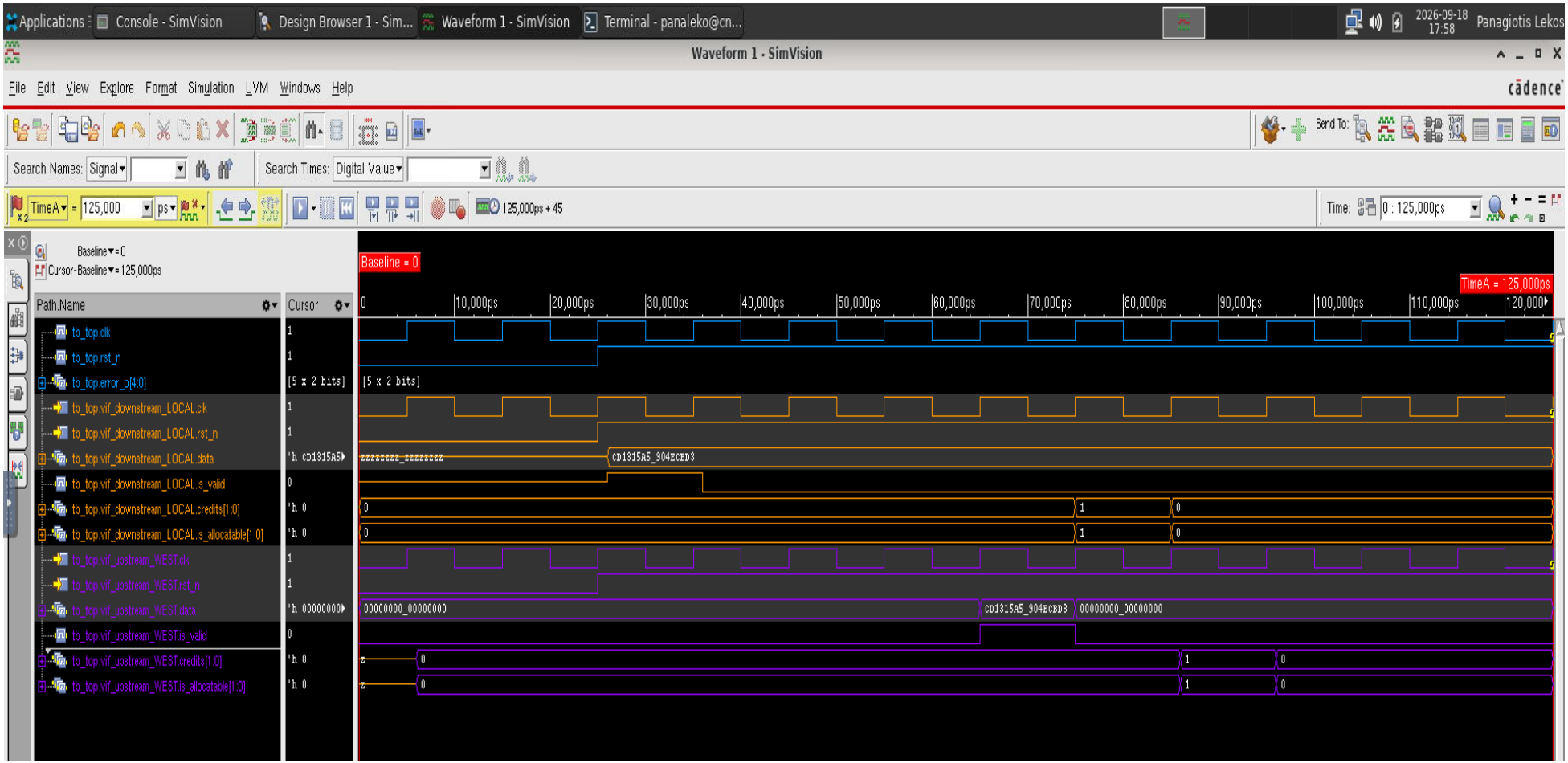
Παρατηρούμε λίγο μετά την ακμή του ρολογιού στα 25ns, αφού ανέλθει και το rst_n, στο LOCAL downstream vif (virtual interface) το σήμα *data* παίρνει την τιμή του flit, και σηκώνεται το σήμα *is_valid* που σημαίνει πώς ο Master Agent έστειλε επιτυχώς το flit.

Στα 65ns στο WEST upstream vif το *data* παίρνει την ίδια τιμή με πριν και σηκώνεται το *is_valid*. Έτσι βλέπουμε πως το DUT έστειλε το flit στη σωστή θύρα.

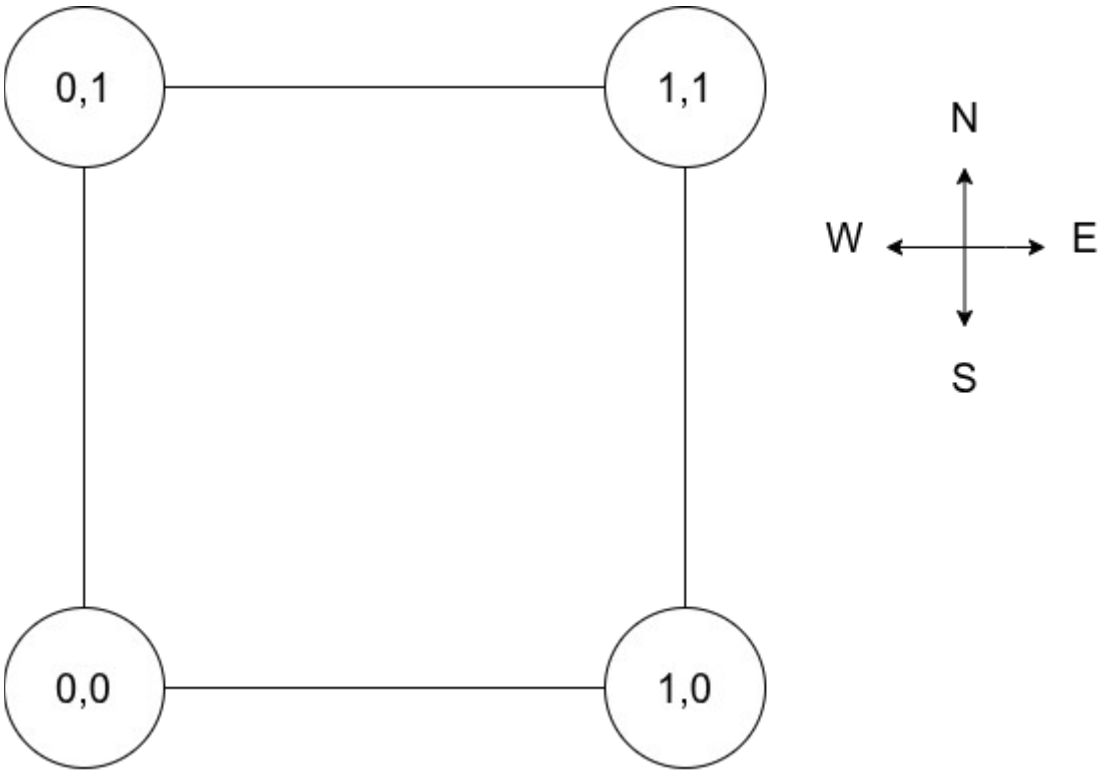
Στα 75ns στο LOCAL downstream vif σηκώνονται τα σήματα *credits, is_allocatable* για έναν κύκλο ρολογιού. Αυτό δείχνει πώς το DUT λειτούργησε σωστά και επέστρεψε αυτά τα σήματα στην θύρα απ’ όπου έλαβε το πακέτο, ακριβώς έναν κύκλο αφότου το έστειλε, σύμφωνα με το backpressure mechanism.

Στα 85ns, τα παραπάνω προηγούμενα σήματα ανέρχονται και στο WEST upstream vif, που δείχνει πώς και ο Slave Agent υλοποιεί σωστά το backpressure mechanism όταν λαμβάνει πακέτα.

Τα αποτελέσματα μας δείχνουν πώς, τουλάχιστον σε αυτό το απλό σενάριο αποστολής και λήψης ενός πακέτου, το testbench, και το DUT λειτουργούν ορθά.



Σχήμα: Κυματομορφές παραγμένες από το Sanity Test



Σχήμα: Τοπολογία πλέγματος του NoC

--- 22/9/2026 ---

Packet Test & Multi Packet Test

Το Sanity Test ήταν μια αρχική επιβεβαίωση της ορθής λειτουργίας του Testbench, αλλά πριν δείξουμε εμπιστοσύνη στο Testbench αποφασίστηκε να γίνουν μερικά ακόμα test, διότι το Sanity Test είναι το πιο απλό σενάριο που υπάρχει. Για αυτόν τον λόγο δημιουργήθηκαν τα Packet Sequence, Multi Packet Sequence, Packet Test και Multi packet Test.

Το Packet Sequence δημιουργεί ένα τυχαίο πακέτο αποτελούμενο από 1 - 4 flits, καθορίζοντας τα label τους με την απαιτούμενη σειρά (πρώτο HEAD, τελευταίο TAIL κλπ). Τα υπόλοιπα μεγέθη (μήκος πακέτου, κανάλι, συντεταγμένες προορισμού) τυχαιοποιούνται ή καθορίζονται από το Test. Το Packet Test στέλνει ένα τυχαιοποιεί και στέλνει πακέτο με την Packet Sequence από τον Local Master Agent, και αφήνει ένα περιθώριο drain time για να προλάβουν να γίνουν όλες οι ενέργειες στο DUT, μέχρι να τελειώσει το simulation time.

Το Multi Packet Sequence χρησιμοποιεί το Packet Sequence για να δημιουργήσει μια ακολουθία από πακέτα. Οι παράμετροι (αριθμός πακέτων, κανάλι, προορισμός) είτε τυχαιοποιούνται είτε παίρνουν τιμές από το Test. Το Multi Packet Test στέλνει αυτήν την ακολουθία πακέτων, από τον Local Master Agent, ορίζοντας συγκεκριμένες τιμές για τις παραμέτρους του.

Επίσης έχει δημιουργηθεί ένα base test το οποίο κάνουν extend όλα τα υπόλοιπα tests. Αυτό που κάνει είναι αν δημιουργεί όλους τους Agents, και να δίνει default τιμές στην καθυστέρηση κατανάλωσης flit από τον slave, κάτι που μπορεί να γίνει overwrite από τα test αν χρειάζεται.

Packet Test Simulation

Logs

../seq/router_packet_seq.sv(30) @ 0: uvm_test_top.env.master_LOCAL.sequencer@@pkt_seq [router_packet_seq] Starting packet: len=4 vc=ESCAPE dest=(0,1)

../env/router_master_driver.sv(98) @ 25000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Driving Flit to DUT: HEAD vc=ESCAPE dest=(0,1) pl=38b03e4fb2a4962

../env/router_monitor.sv(37) @ 35000: uvm_test_top.env.master_LOCAL.monitor [router_monitor] Captured Flit: HEAD vc=ESCAPE dest=(0,1) pl=38b03e4fb2a4962

../env/router_master_driver.sv(98) @ 35000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Driving Flit to DUT: BODY vc=ESCAPE dest=(0,0) pl=43d5300444a768

../env/router_monitor.sv(37) @ 45000: uvm_test_top.env.master_LOCAL.monitor [router_monitor] Captured Flit: BODY vc=ESCAPE dest=(0,0) pl=43d5300444a768

../env/router_master_driver.sv(98) @ 45000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Driving Flit to DUT: BODY vc=ESCAPE dest=(0,0) pl=179f8b57773deeaf

../env/router_monitor.sv(37) @ 55000: uvm_test_top.env.master_LOCAL.monitor [router_monitor] Captured Flit: BODY vc=ESCAPE dest=(0,0) pl=179f8b57773deeaf

../env/router_master_driver.sv(98) @ 55000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Driving Flit to DUT: TAIL vc=ESCAPE dest=(0,0) pl=35fae294d7a4f5d

../seq/router_packet_seq.sv(53) @ 55000: uvm_test_top.env.master_LOCAL.sequencer@@pkt_seq [router_packet_seq] Packet finished

../tests/router_base_test.sv(59) @ 55000: uvm_test_top [router_packet_test] Stimulus finished, draining...

../env/router_monitor.sv(37) @ 65000: uvm_test_top.env.master_LOCAL.monitor [router_monitor] Captured Flit: TAIL vc=ESCAPE dest=(0,0) pl=35fae294d7a4f5d

../env/router_monitor.sv(37) @ 75000: uvm_test_top.env.slave_WEST.monitor [router_monitor] Captured Flit: HEAD vc=ESCAPE dest=(0,1) pl=38b03e4fb2a4962

../env/router_monitor.sv(37) @ 85000: uvm_test_top.env.slave_WEST.monitor [router_monitor] Captured Flit: BODY vc=ESCAPE dest=(0,0) pl=43d5300444a768

../env/router_slave_driver.sv(91) @ 85000: uvm_test_top.env.slave_WEST.slave_drv [router_slave_driver] Returning 1 Credit to VC ESCAPE

../env/router_master_driver.sv(162) @ 85000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Received 1 Credit from DUT for VC ESCAPE (available: 1/4)

../env/router_monitor.sv(37) @ 95000: uvm_test_top.env.slave_WEST.monitor [router_monitor] Captured Flit: BODY vc=ESCAPE dest=(0,0) pl=179f8b57773deeaf

../env/router_master_driver.sv(162) @ 95000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Received 1 Credit from DUT for VC ESCAPE (available: 2/4)

../env/router_monitor.sv(37) @ 105000: uvm_test_top.env.slave_WEST.monitor [router_monitor] Captured Flit: TAIL vc=ESCAPE dest=(0,0) pl=35fae294d7a4f5d

../env/router_slave_driver.sv(91) @ 105000: uvm_test_top.env.slave_WEST.slave_drv [router_slave_driver] Returning 1 Credit to VC ESCAPE

../env/router_master_driver.sv(162) @ 105000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Received 1 Credit from DUT for VC ESCAPE (available: 3/4)

../env/router_master_driver.sv(162) @ 115000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] Received 1 Credit from DUT for VC ESCAPE (available: 4/4)

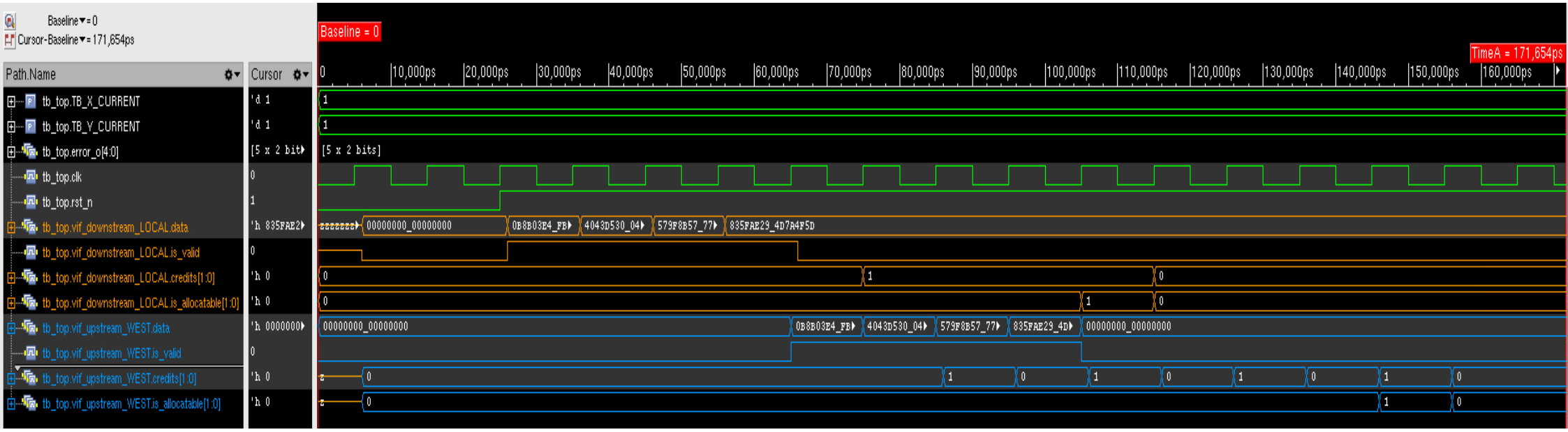
../env/router_master_driver.sv(173) @ 115000: uvm_test_top.env.master_LOCAL.master_drv [router_master_driver] DUT VC ESCAPE is allocatable again

../env/router_slave_driver.sv(91) @ 125000: uvm_test_top.env.slave_WEST.slave_drv [router_slave_driver] Returning 1 Credit to VC ESCAPE

../env/router_slave_driver.sv(91) @ 145000: uvm_test_top.env.slave_WEST.slave_drv [router_slave_driver] Returning 1 Credit to VC ESCAPE

../env/router_slave_driver.sv(99) @ 145000: uvm_test_top.env.slave_WEST.slave_drv [router_slave_driver] Asserting is_allocatable for VC ESCAPE

Waveforms



Παρατηρώντας τις κυμματομορφές και τα logs βλέπουμε:

- ο Master Agent στέλνει ένα πακέτο 4 flits στο DUT (θύρα LOCAL downstream) τις στιγμές 25ns, 35ns, 35ns, 45ns
- Το DUT λαμβάνει αυτά τα flit στην LOCAL downstream θύρα του μετά από έναν κύκλο (λόγω χρήσης clocking block) τις στιγμές 35ns, 45ns, 55ns, 65ns
- Το DUT στέλνει τα flits στον Slave Agent (WEST upstream) τις στιγμές 65ns, 75ns, 85ns, 95ns ((1,1) → (0,1) σωστό)
- Το DUT σε κάθε κύκλο που στέλνει flit στον Slave Agent (WEST upstream), στέλνει και credit στον Master. Μαζί με το τελευταίο credit στέλνει και το is_allocatable
- Ο Slave Agent στέλνει credits στο DUT (WEST upstream) με καθυστέρηση ενός κύκλου, μεταξύ των κάθε flit. Με το τελευταίο flit στέλνει και is_allocatable

--- 27/9/2026 ---

Scoreboard

Το testbench μέχρι στιγμής είναι ικανό να στέλνει flits στο DUT και να δέχεται από αυτό σήματα ελέγχου (μέσω των Master Agents), να δέχεται flits από το DUT και να στέλνει σε αυτό σήματα ελέγχου (Slave Agents), να παρακολουθεί την κίνηση των flits στις εισόδους και εξόδους του DUT (Monitors). Όμως δεν υπάρχει αυτόματος έλεγχος ώστε να ελεγχθεί η σωστή λειτουργία του DUT, καθώς για να γίνει αυτό πρέπει να ελεγχθούν “με το μάτι” η κυμματομορφές και τα logs που παράγονται από το εκτέλεση των tests. Αυτό το πρόβλημα έρχεται να λύσει το Scoreboard, ένα UVM component. Σκοπός του είναι να λαμβάνει όλα τα transactions από τους monitors (εισόδου και εξόδου), και να ελέγχει αν το DUT συμπεριφέρεται σύμφωνα με τον σχεδιασμό του.

Πιο συγκεκριμένα το Scoreboard πρέπει να ελέγχει τα παρακάτω:

- Κάθε πακέτο πρέπει να εξέρχεται από την σωστή κατεύθυνση εξόδου σύμφωνα με τις συντεταγμένες προορισμού του, αλλά και το είδος δρομολόγησής του, το οποίο εξαρτάται από το κανάλι δρομολόγησης.
- Κάθε πακέτο πρέπει να εξέρχεται από νόμιμο κανάλι εξόδου με βάση το κανάλι εισόδου του, σύμφωνα με το πρωτόκολλο.
- Τα δεδομένα κάθε flit (label, (x_dest, y_dest), payload) και κάθε πακέτου πρέπει να λαμβάνονται αναλλοίωτα στην έξοδο.
- Πρέπει τα flits ενός πακέτου να λαμβάνονται με τη σειρά που στάλθηκαν.
- Πρέπει όλα τα flits που εισήλθαν στο DUT, αυτά να εξέλθουν από αυτό, κανένα περισσότερο κανένα λιγότερο.

Εναλλαγή εικονικού καναλιού

Το DUT μπορεί να αλλάξει το εικονικό κανάλι ενός flit, κάνοντας overwrite το bit vc_id, αλλά μόνο από ADAPTIVE σε ESCAPE. Επομένως τα νόμιμα ζευγάρια VC εισόδου/εξόδου είναι (ADAPTIVE → ADAPTIVE), (ESCAPE → ESCAPE), (ADAPTIVE → ESCAPE), ενώ το ζευγάρι (ESCAPE → ADAPTIVE) είναι παράνομο και το Scoreboard πρέπει να μπορεί να το εντοπίζει και το αναφέρει.

Επιτρεπόμενη δρομολόγηση

Υπάρχουν δύο τύποι δρομολόγησης που επιτρέπει το DUT, η προσαρμοστική δρομολόγηση και το XY routing.

Προσαρμοστική δρομολόγηση: Οι επιτρεπόμενες κατευθύνσεις εξόδου είναι αυτές που μειώνουν την απόσταση από τον παρόντα κόμβο στον κόμβο προορισμού. Στην περίπτωση που οι συντεταγμένες διαφέρουν σε δύο διαστάσεις οι νόμιμες έξοδοι είναι δύο, ενώ όταν διαφέρουν σε μία υπάρχει μία νόμιμη έξοδος. Αν οι συντεταγμένες αυτές συμπίπτουν τότε η θύρα εξόδου είναι η LOCAL.

XY Routing: Σε αυτήν την δρομολόγηση, έχει προτεραιότητα η μείωση της απόστασης κατά X. Αν οι συντεταγμένες παρόντος κόμβου και κόμβου προορισμού διαφέρουν κατά X υπάρχει μία νόμιμη κατεύθυνση εξόδου κατά X (WEST ή EAST). Αν κατά X είναι ίδιες, τότε αν διαφέρουν κατά Y τότε οι νόμιμη έξοδος είναι μόνο κατά Y (NORTH ή SOUTH). Ξανά αν είναι ίδιες κατά X και Y, η έξοδος είναι η LOCAL. Σε αυτήν την δρομολόγηση πάντα η νόμιμη έξοδος είναι μία και ποτέ παραπάνω.

Όταν ένα flit έχει κανάλι εξόδου ADAPTIVE, η κατεύθυνση εξόδου υπολογίζεται με προσαρμοστική δρομολόγηση, ενώ όταν το κανάλι εξόδου είναι ESCAPE, υπολογίζεται με XY Routing. Το Scoreboard πρέπει να ελέγχει αν η κατεύθυνση εξόδου των πακέτων έχει υπολογιστεί σωστά σύμφωνα με την κατεύθυνση εισόδου και το κανάλι εξόδου, και να αναφέρει έχει υπάρξει λάθος.

Αρχιτεκτονική του Scoreboard

Το Scoreboard λαμβάνει τα flits από τους 10 Monitor, 5 Master και 5 Slave. Οι Master καταγράφουν τα flit εισόδου στο DUT, ενώ οι Slave καταγράφουν τα flits εξόδου από το DUT, και τα στέλνει το κάθε ένα σε ένα analysis port. Το Scoreboard περιέχει δύο ports, τα in (Masters) και τα out (Slaves), για να δέχεται transactions. Για αυτό θα χρειαστεί να προστεθεί ένα ακόμα πεδίο στο sequence item, η κατεύθυνση εισόδου, ώστε η πληροφορία “από πού ήρθε” το flit να περιέχεται μέσα στα transactions που λαμβάνει το Scoreboard. Αυτό το πεδίο θα προστίθεται από τους Monitors που καταγράφουν τα flits.

Τα flits που λαμβάνονται ομαδοποιούνται σε πακέτα. Μια νέα εγγραφή πακέτου αρχίζει με τη λήψη HEAD (ή HEADTAIL) flit, προστίθενται τα BODY flits αν υπάρχουν, και κλείνει με TAIL (ή HEADTAIL). Τα πακέτα εισόδου (που εισέρχονται στο DUT) αποθηκεύονται σε έναν πίνακα για να γνωρίζει το Scoreboard ποια πακέτα βρίσκονται εντός του DUT και δεν έχουν εξέλθει ακόμα. Όταν λαμβάνονται πακέτα εξόδου το Scoreboard πρέπει να ελέγχει το payload (και label και συντεταγμένες προορισμού?) και να ψάχνει να το αντιστοιχίζει με πακέτα του παραπάνω πίνακα. Εφόσον βρεθεί το ζεύγος εισόδου-εξόδου πρέπει να ελεγχθεί αν τηρούνται οι κανόνες δρομολόγησης και εναλλαγής καναλιού που αναφέρθηκαν προηγουμένως, και αν υπάρχει κάποιο πρόβλημα να αναφερθεί σαν σφάλμα. Αφού γίνουν αυτά, το πακέτο αφαιρείται από τον πίνακα, καθώς εξήλθε απ’ το DUT. Αν όμως δεν καταφερθεί να αντιστοιχηθεί πακέτο εξόδου με εισόδου, πρέπει κι αυτό να αναφερθεί σαν σφάλμα καθώς σημαίνει πως είτε κάποιο flit αλλοιώθηκε εντός του DUT, είτε το DUT έστειλε κάποιο flit χωρίς να το έχει λάβει. Στο τέλος του simulation ελέγχεται αν υπάρχουν εναπομείναντα πακέτα στον πίνακα. Αν ναι, σημαίνει ότι υπήρξαν πακέτα που εισήλθαν στο DUT και δεν εξήλθαν ποτέ από αυτό και πρέπει να αναφερθεί σαν σφάλμα.

--- 6/10/2026 ---

Δομή κώδικα Scoreboard

Αρχικά δημιουργούμε μια βοηθητική κλάση router_sb_packet, η οποία περιέχει πληροφορίες πακέτου, για τη διαχείρισή του εντός του Scoreboard. Τα πεδία της περιέχουν την θύρα και το VC εισόδου του πακέτου στο DUT, τα flits που το αποτελούν, αν το πακέτο εντοπίστηκε στην έξοδο του DUT, και μια μεταβλητή που δείχνει κάθε στιγμή ποιο είναι το επόμενο flit που αναμένεται να εξέλθει απ' το DUT.

Η κλάση scoreboard περιέχει δύο imps, απ' όπου δέχεται τα flits που στέλνει ο monitor, το imp_in δέχεται τα flit που εισέρχονται στο DUT, και το imp_out αυτά που εξέρχονται από το DUT. Τα dut_x, dut_y δείχνουν την θέση στο πλέγμα του router DUT. Ο πίνακας in_flight περιέχει handles στα πακέτα που βρίσκονται εντός του του DUT (έχουν εισέλθει αλλά δεν έχουν εξέλθει). Οι πίνακες in_open, out_open βοηθούν στο χτίσιμο των πακέτων καθώς τα flit εισέρχονται και εξέρχονται ένα ένα από το DUT.

Η συνάρτηση write_in καλείται κάθε φορά που ο monitor παρατηρεί ένα flit να εισέρχεται στο DUT. Για flit τύπου HEAD/HEADTAIL, αρχικά ελέγχει αν στη θέση του (θύρα, vc), υπάρχει ανοιχτό πακέτο. Αν ναι έχει γίνει σφάλμα, καθώς ανοίγει νέο πακέτο χωρίς να έχει κλείσει προηγούμενο. Δημιουργεί νέο πακέτο θέτοντας τα (θύρα, vc) του και δίνονται handles για αυτό στον in_flight και στον in_open στην αντίστοιχη θέση. Αν το εισερχόμενο flit είναι BODY/TAIL, ελέγχεται αν υπάρχει ανοιχτό πακέτο σε αυτή τη θέση, και αν όχι, υπάρχει σφάλμα καθώς ξεκινάει νέο πακέτο με BODY/TAIL αντί για HEAD/HEADTAIL. Μετά, κάθε εισερχόμενο flit ανεξαρτήτου label προστίθεται στο πακέτο. Αν είναι TAIL/HEADTAIL το handle του in_open σταματάει να δείχνει στο πακέτο.

Η write_out εκτελείται όταν εξέρχεται κάποιο flit από το DUT. Αν το εξερχόμενο flit είναι HEAD/HEADTAIL ελέγχεται αν υπάρχει ανοιχτό πακέτο σε αυτήν την θέση (θύρα, vc). Αν ναι υπάρχει σφάλμα. Μέσω της find_match() εξετάζεται αν υπάρχει πακέτο στον in_flight με HEAD/HEADTAIL flit ίδιο με το εξερχόμενο flit είναι. Αν δεν βρεθεί έχουμε σφάλμα. Εφόσον βρεθεί το πακέτο δηλώνεται αντιστοιχισμένο και τίθεται ο δείκτης next_out στο δεύτερο flit. Με την check_routing() ελέγχεται ο ορθότητα της δρομολόγησης του πακέτου. Δίνεται handle του αντιστοιχισμένου πακέτου στο out_open στην αντίστοιχη θέση. Αν το εξερχόμενο flit είναι BODY/TAIL, ελέγχεται αν υπάρχει ανοιχτό πακέτο σε αυτή τη θέση, αν όχι υπάρχει σφάλμα. Στη συνέχεια γίνονται δύο ακόμα έλεγχοι, αν ο δείκτης επόμενου flit είναι εντός ορίων, κι εφόσον ναι, αν είναι ίδιο το επόμενο flit του πακέτου με το εξερχόμενο flit. Εδώ αυξάνεται ο δείκτης επόμενου flit του πακέτου. Στο τέλος για εξερχόμενα flits τύπου TAIL/HEADTAIL αφαιρείται το handle από τον out_open στη θέση αυτή, και από τον in_flight.

--- 10/10/2026 ---

Εύρεση σφάλματος δρομολόγησης

Εκτέλεση Sanity Test με Scoreboard

Αφού αναπτύχθηκε το Scoreboard, το αμέσως επόμενο βήμα είναι να εκτελεσθεί το sanity test. Υπενθυμίζω πως το sanity test μέσω του LOCAL agent στέλνει στην downstream θύρα του DUT ένα flit με:

- flit_label = HEADTAIL
- vc_id = ESCAPE
- (x_dest, y_dest) = (0,0)
- payload = random

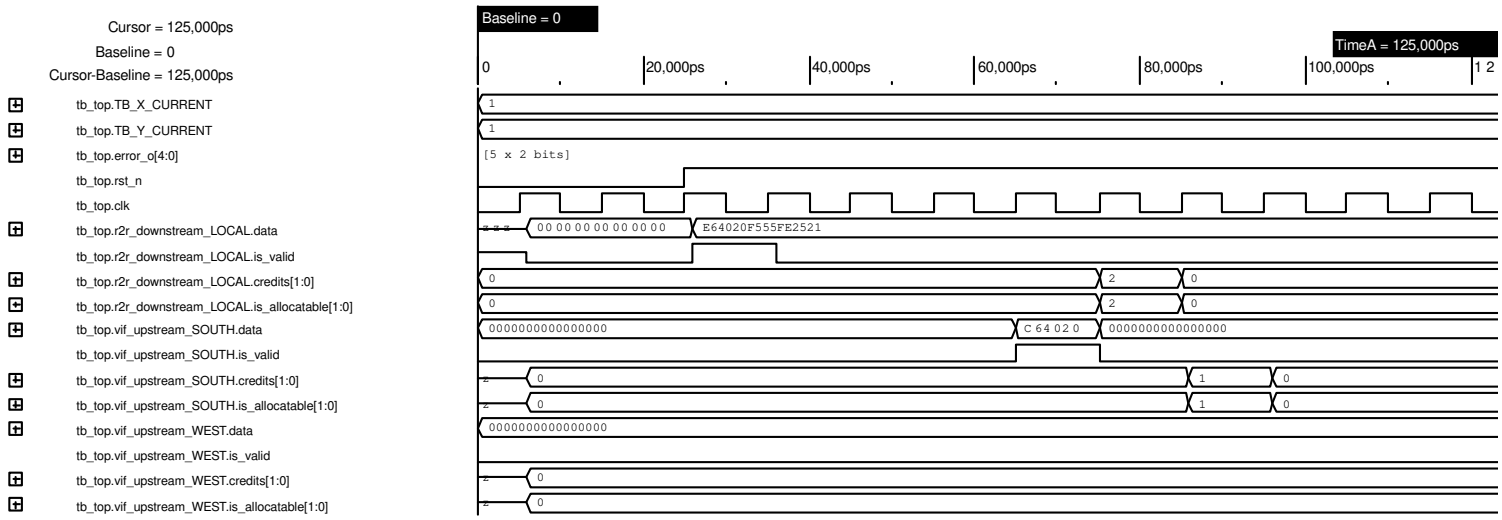
και το test εκτελείται με συντεταγμένες του DUT (1,1)

Το test περνάει και δεν εντοπίζονται σφάλματα.

Ένα ενδιαφέρον εύρημα ανακαλύφθηκε, αν στο sanity sequence αλλαχθεί το vc_id σε ADAPTIVE. Τότε το Scoreboard αναφέρει το παρακάτω σφάλμα:

UVM_ERROR ../env/router_scoreboard.sv(136) @ 75000: uvm_test_top.env.sb [SB_ROUTE] ESCAPE output not on XY port (WEST): [SOUTH] HEADTAIL vc=ESCAPE dest=(0,0) pl=64020f555fe2521

Το flit ενώ άλλαξε vc_id από ADAPTIVE σε ESCAPE (που είναι επιτρεπτό από τα specs) δρομολογήθηκε στην θύρα SOUTH, η οποία μπορεί να συμφωνεί με προσαρμοστική δρομολόγηση αλλά όχι με XY δρομολόγηση.



--- 12/10/2026 ---

Επικοινωνία με τον Designer και διευκρίνηση των specs

Το εύρημα αυτό γνωστοποιήθηκε στον designer, και μετά από συζήτηση και διαλεύκανση του θέματος, η συμπεριφορά αυτή του κυκλώματος θεωρήθηκε αποδεκτή, και σχεδιαστική επιλογή.

Επομένως αποσαφηνίστηκε πώς η δρομολόγηση ενός flit από το DUT εξαρτάται από το vc_id με το οποίο το flit εισήλθε στο DUT και όχι από αυτό με το οποίο εξήλθε. Έτσι, στην περίπτωση που το DUT μετατρέπει το vc_id από ADAPTIVE σε ESCAPE, αυτό το flit δρομολογείται με προσαρμοστική δρομολόγηση, και αν το DUT βρισκόταν σε ένα πλέγμα NoC, η XY δρομολόγηση θα ξεκινούσε από τον επόμενο router.

--- 13/10/2026 ---

Αλλαγή στο Scoreboard

Πλέον το είδος της δρομολόγησης του flit εξαρτάται από το vc_id εισόδου του, αντί το εξόδου. Επομένως το Scoreboard πρέπει να προσαρμοστεί ανάλογα. Έτσι πλέον η συνάρτηση check_routing() που ελέγχει την ορθότητα των δρομολογήσεων, το κάνει με βάση το vc_id του flit εισόδου, αντί για εξόδου.

Μετά την αλλαγή αυτή στο Scoreboard η προσομοίωση τρέχει χωρίς errors.

--- 14/10/2026 ---

Δεύτερη Επικοινωνία με τον Designer

Μετά από επικοινωνία με τον Designer τα specs επέστρεψαν στην πρώτη μορφή. Επομένως παραμένουμε στο ότι η συμπεριφορά του DUT που εντοπίστηκε είναι σφάλμα, η διομολόγηση γίνεται με βάση το flit εξόδου από το DUT, το Scoreboard επαναφέρεται στην αρχική μορφή και αναμένεται να διορθωθεί το Design.

--- 18/10/2026 ---

Τρίτη Επικοινωνία με τον Designer

Διορθώθηκε η RTL, επομένως τώρα στην περίπτωση που ο router αλλάζει το vc_id ενός flit από ADAPTIVE σε ESCAPE, τότε δρομολογείται σε θύρα που συμφωνεί με την δρομολόγηση των ESCAPE καναλιών, δηλαδή την XY.

Επίσης έγινε μια ακόμη αλλαγή, στον τρόπο επιλογής εικονικού καναλιού. Όταν ένα flit με vc_id = ADAPTIVE εισέρχεται στον router τότε, εφόσον υπάρχουν διαθέσιμα ADAPTIVE κανάλια, θα δρομολογηθεί σε αυτά και όχι σε ESCAPE. Αν επαναλαμβανόμενα δεν βρίσκονται credits ώστε να σταλεί το flit, τότε μετά από έναν συγκεκριμένο αριθμό προσπαθειών ο οποίος καθορίζεται από μια μεταβλητή ADAPTIVE_TIMEOUT, το flit υποβιβάζεται σε ESCAPE κανάλι. Στην συγκεκριμένη υλοποίηση το ADAPTIVE_TIMEOUT ισούται με 5.

Εκτέλεση των υπαρχόντων test μετά τις αλλαγές στην RTL

Μετά τις αλλαγές στο Design έπρεπε να εκτελεστούν ξανά τα υπάρχοντα test για να επιβεβαιωθεί ότι συνεχίζουν να περνάνε και δεν τροποποιήθηκε κάτι που δεν έπρεπε. Όλα τα test (sanity_test, packet_test ,multi_packet_test) εκτελέσθηκαν επιτυχώς.

Πρέπει όμως να σημειωθεί ότι τώρα πια στο sanity test δεν συμβαίνει ο υποβιβασμός από ADAPTIVE σε ESCAPE, καθώς το DUT βρίσκει ελεύθερο ADAPTIVE κανάλι, και δρομολογεί το flit σε αυτό, σύμφωνα με τις νέες βελτιώσεις του Design. Έτσι όμως δεν μπορεί να ελεγχθεί αν εξαλείφθηκε το bug που εντοπίστηκε προηγουμένως. Για αυτό θα χρειαστεί στοχευμένο test, όπου οι slave agents δεν θα ελευθερώνουν τα ADAPTIVE κανάλια τους, κι έτσι θα αναγκαστεί το DUT να υποβιβάσει τα flits που δρομολογεί από ADAPTIVE σε ESCAPE.